

Hackathon Project Phases Template is structured to capture essential information without being time-consuming.

Hackathon Project Phases Template

- **PROJECT NAME:**

AI personalized email generator

- **TEAM NAME:**

comates

- **Team Members:**

- 1) D.NAGA CHANDRIKA

- 2) E.KUSUMA

Chapter 1 Phase-1: Brainstorming & Ideation

Objective:

- ❖ **Problem statement:**

Without an email generator, individuals and businesses face several challenges that hinder the effectiveness and efficiency of email communication. Writing emails manually is time-consuming, especially when dealing with large volumes, leading to inefficiencies and delays. Personalization becomes difficult, resulting in generic messages that fail to engage recipients, lowering response rates and customer satisfaction. The quality and tone of emails can be inconsistent, causing confusion or misinterpretation, while the risk of errors such as typos, grammatical mistakes, and missed details increases. Moreover, scaling email communication for large audiences becomes a daunting task, and crafting compelling subject lines or following up effectively is often neglected.

Additionally, without proper segmentation, emails may lack relevance for recipients, further reducing their impact. In summary, without an email generator, individuals struggle with time constraints, personalization, error prevention, and consistency, leading to ineffective communication and missed opportunities.

❖ Define the purpose and impact of the project:

- **Impact of AI-Generated Emails:** AI-generated emails significantly enhance the efficiency and effectiveness of communication by automating the process of writing personalized, high-quality messages. They help reduce the time and effort spent on manual email creation, improve consistency in tone and content, and ensure error-free communication. With the ability to tailor messages based on recipient data, AI-generated emails drive higher engagement, increase open rates, and boost customer satisfaction. They also enable better scalability for businesses, allowing them to reach large audiences with personalized content.
- **Purpose of AI-Generated Emails:** The primary purpose of AI-generated emails is to automate and streamline email communication, ensuring that messages are personalized, relevant, and impactful. By leveraging artificial intelligence, businesses can deliver timely, tailored content at scale, improve customer relationships, and increase conversion rates, all while saving time and resources.
-

❖ Key Points:

1. **Problem Statement:** EMAIL GENERATOR USING AI
 2. **Proposed Solution:** The AI personalized email streamlines the email drafting process by allowing users to input specific details such as recipients name, event name and special instructions. It then generates a customized Email body according to the users input.
 3. **Target Users:** Sales people, marketing professionals, Business owners and any one who needs to send a large volume of personalized events effectively
 4. **Expected Outcome:**
 - Time-saving
 - Personalization Consistency
 - Error-free
 - Scalability
 - Cost-effective
 - Improved engagement
 - Data-driven
 - Customizable tone
-

Chapter 2 Phase-2: Requirement Analysis

❖ Objective:

- Define technical and functional requirements.
- **Natural Language Processing (NLP):** Ability to analyze and generate human-like text based on user inputs and recipient data.

Machine Learning Algorithms: For learning from user data, recipient behavior, and past email performance to generate optimized content.

Personalization: Automatically customizes content based on recipient data.

Template Selection: Ability to choose or generate various email templates based on campaign goals.

Customizable Tone and Style: Allows users to select the tone (formal, casual, etc.) and style of communication.

❖ Key Points:

1. **Technical Requirements:** python,NLP,machine learning algorithms etc..
 2. **Functional Requirements:** personalization,tempelate selection etc..
 3. **Constraints & Challenges:** Data Privacy and Security, Context Understanding, Content Accuracy etc..
-

Chapter 3 Phase-3: Project Design

❖ Objective:

- Create the architecture and user flow.
- Automate Email Personalization, streamline User Experience,etc..



❖ Key Points:

1. System Architecture Diagram: email-generator/

|

| — backend/

| | — app.py # Main Flask app with routing and API endpoints

| | — ai_engine.py # AI model for generating email content

| | — email_service.py # Handles email delivery (SMTP/SendGrid integration)

| | — database.py # Manages database operations

| | — templates/ # Predefined email templates (HTML/CSS)

| | — utils.py # Utility functions (e.g., data cleaning, tokenization)

|

| — frontend/

| | — src/

| | | — App.js # Main React component for UI

| | | — EmailForm.js # Form for composing and customizing email

| | | — api.js # API calls to backend

| | — public/

```

|   └─ index.html # Main HTML file
|
|   └─ models/ # AI models and training scripts
|
|   └─ email_model.py # Fine-tuned GPT-3 or custom model for email generation
|
|   └─ requirements.txt # Project dependencies
|
|   └─ README.md # Project documentation

```

2. **User Flow:** The user will interact with the AI Personalized Email Generator through a simple web interface. Here's how it works:

Input Email Details: The user enters the subject, body content (or selects from predefined templates), and a list of recipients (e.g., in a comma-separated format).

Customize Email: The user can choose additional options, like the tone of the email (formal, casual) or specific personalization settings.

Generate Email: Once the user submits the form, the backend AI engine processes the input data, personalizes the email for each recipient using AI, and generates the final content.

Preview and Edit: The user can preview the personalized emails and make any adjustments if necessary.

Send Email: The user triggers the email sending, either immediately or by scheduling for a later time.

Track Performance: After sending, the user can view metrics (like open rates or click-through rates) to track the success of the email campaign.

This process simplifies email marketing or communication by automating content creation and ensuring personalization at scale.

3. **UI/UX Considerations:** Simplicity and Clarity, Personalization Options etc..

Chapter 4 Phase-4: Project Planning (Agile Methodologies)

❖ Objective:

- Break down the tasks using Agile methodologies.

Sprint	Task	Priority	Duration	Deadline	Assigned To	Dependencies	E
Sprint 1	Environment Setup & API	● High	6 hours (Day 1)	End of Day 1	Member 1	Google API Key, Python,	C

	Integration					Streamlit setup	
Sprint 1	Frontend UI Development	□ Medium	2 hours (Day 1)	End of Day 1	Member 2	API response format finalized	B
Sprint 2	Vehicle Search & Comparison	● High	3 hours (Day 2)	Mid-Day 2	Member 1& 2	API response, UI elements ready	S
Sprint 2	Error Handling & Debugging	● High	1.5 hours (Day 2)	Mid-Day 2	Member 1&4	API logs, UI inputs	Ir
Sprint 3	Testing & UI Enhancements	□ Medium	1.5 hours (Day 2)	Mid-Day 2	Member 2& 3	API response, UI layout completed	
Sprint 3	Final Presentation & Deployment	□ Low	1 hour (Day 2)	End of Day 2	Entire Team	Working prototype	D

1. **Task Allocation:** team contain 2 members both the persons will have the equal allocation
2. **Timeline & Milestones:** by the end of the day

Chapter 5 Phase-5: Project Development

❖ Objective:

- Code the project and integrate components.

❖ Key Points:

1. **Technology Stack Used:** (List of programming languages, APIs, etc.)
 - **Frontend:** Streamlit
 - **Backend:** Google Gemini Flash API
 - **Programming Language:** Python
2. **Development Process:** (Steps followed for coding)
 - Implement **API key authentication** and **Gemini API integration**.
 - Develop **vehicle comparison and maintenance tips logic**. ◦ Optimize **search queries for performance and relevance**.
3. **Challenges & Fixes:** (Mention any obstacles faced and how they were solved)
 - **Challenge:** Delayed API response times.
Fix: Implement **caching** to store frequently queried results.
 - **Challenge:** Limited API calls per minute.
Fix: Optimize queries to fetch **only necessary data**.

Chapter 6 Phase-6: Functional & Performance Testing

❖ Objective:

- Ensure the project works as expected.

4. Test Case ID	Category	Test Scenario	Expected Outcome	Status	Tester
TC-001	Functional Testing	Query "Best budget cars under ₹10 lakh"	Relevant budget cars should be displayed.	✔ Passed	Tester 1
TC-002	Functional Testing	Query "Motorcycle maintenance tips for winter"	Seasonal tips should be provided.	✔ Passed	Tester 2
TC-003	Performance Testing	API response time under 500ms	API should return results quickly.	⚠ Needs Optimization	Tester 3

TC-004	Bug Fixes & Improvements	Fixed incorrect API responses.	Data accuracy should be improved.	☑ Fixed	Developer
TC-005	Final Validation	Ensure UI is responsive across devices.	UI should work on mobile & desktop.	✗ Failed - UI broken on mobile	Tester 2
TC-006	Deployment Testing	Host the app using Streamlit Sharing	App should be accessible online.	🚀 Deployed	DevOps

•

❖ Key Points:

1. **Test Cases Executed:** (List the scenarios tested)
2. **Bug Fixes & Improvements:** (Mention fixes made)
3. **Final Validation:** (Does the project meet the initial requirements?)
4. **Deployment (if applicable):** (Hosting details or final demo link)

Chapter 7 Final Submission

1. **Project Report Based on the templates**
 2. **Demo Video (3-5 Minutes)**
 3. **GitHub/Code Repository Link**
 4. **Presentation**
-